Event Sourcing

store what happened, derive what is

Watch the 4-min video

In one line: event sourcing stores every change to an entity as an immutable event in an append-only stream and derives current state by replaying them; you gain a complete audit trail, replay and time travel, and you pay with schema evolution, snapshots and separate read models for queries.

1 How did the balance get here?

A customer calls: 'My balance is wrong; how did it get to this number?' Your accounts table stores one value, the current balance. Every update overwrote the last one, so the history is gone. Accountants solved this centuries ago with a **ledger**: you never erase, you only add entries. Event sourcing applies that idea to software.

2 How it works

Each entity (an account, an order) has an **event stream**: an ordered list of facts such as `Deposited` or `Withdrew`. Events are **immutable**, only appended. To get the current state, load the stream and apply the events in order. To change state, a command loads the state, checks the business rules, and appends new events.

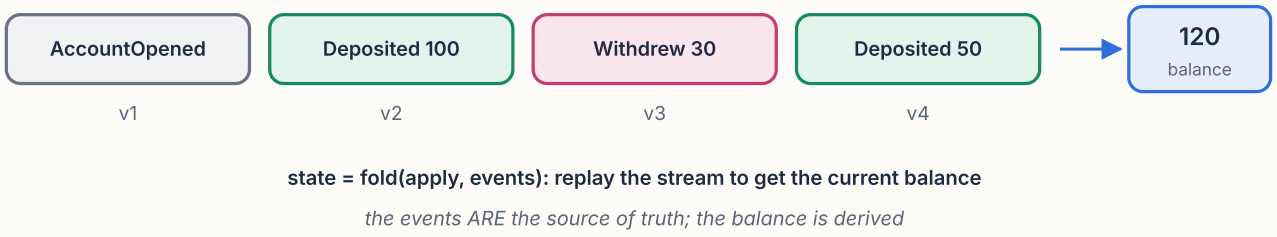


Figure 1. The balance is computed from the events, not stored as the truth.

WORKED EXAMPLE: A BANK ACCOUNT BALANCE

The stream for account 7 holds `AccountOpened`, `Deposited 100`, `Withdrew 30`, `Deposited 50`: balance 120 at version 4. A request to withdraw 200 loads the stream, sees 120, and is rejected. A request to withdraw 20 appends `Withdrew 20` with **expected version 4**. If another withdrawal appended version 5 in the meantime, this append fails with a concurrency error, and the command reloads and re-checks, so two withdrawals can't both pass a balance check that only one would. When the customer calls, support lists the events and shows exactly how 100 became 120.



a concurrent writer that appended v5 first makes this append fail → reload and retry

Figure 2. Optimistic concurrency with an expected version keeps business rules safe.

3 Practical pieces

- **Event store:** EventStoreDB, or a Postgres table with (stream_id, version) unique, or Kafka plus a database for the per-stream check.
- **Snapshots:** for long streams, store the state every N events and replay only the rest.
- **Projections:** to query 'all accounts with balance below 0', build read models from the events (CQRS, lesson 36).
- **Versioning:** old events never change, so readers must understand every old version (upcasting).
- **Personal data:** 'right to be forgotten' needs crypto-shredding or keeping PII out of events.

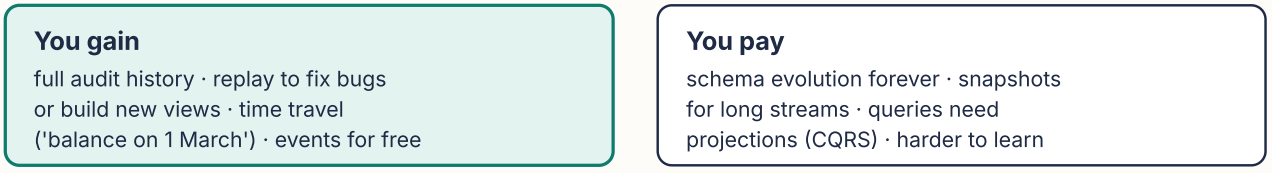


Figure 3. Event sourcing's trade-off. Use it for domains where history matters.

COMMON MISTAKES

- Event sourcing every entity in the system, including simple CRUD.
- No expected-version check, so concurrent commands break rules.
- Querying the event store directly instead of building projections.
- Editing or deleting old events.

4 Key terms

Event store: append-only storage of event streams

Event stream: all events of one entity, in order

Rehydration: rebuilding state by replaying events

Snapshot: saved state to shorten replay

Projection: read model built from events

Expected version: optimistic concurrency check on append

5 Interview questions

Q1. What is event sourcing? BASIC

Persisting each change as an immutable event and deriving current state by replaying the events, instead of storing and overwriting the current state.

Q2. What are its main benefits? BASIC

A complete audit trail, the ability to replay events to fix bugs or build new views, time travel to past states, and a natural stream of events for other services.

Q3. Two withdrawals arrive at once for an account with 120. How do you prevent overdraft? SCENARIO

Both load version 4; each appends with expected version 4; one succeeds, the other gets a concurrency error, reloads, sees the new balance and is rejected if funds are short.

Q4. An account has 2 million events and loads slowly. Fix it. SCENARIO

Store snapshots every N events and replay only events after the latest snapshot.

Q5. How do you change the structure of an event that already exists in the store? TRICKY

You don't edit stored events; you add a new version and upcast old ones when reading, or write a migration to a new stream.

Q6. When is event sourcing a bad idea? TRICKY

For simple CRUD data with no need for history, teams new to it, or domains where querying current state is all that matters; the complexity isn't repaid.

Next: SD-I-014 · CQRS